



Sesión 09-09-2026: Delegación multi-proveedor

➤ Trazabilidad append-only + delegación determinista a subagentes, inspirada en el DeepSeek Harness

Resumen ejecutivo

Durante esta mañana se incorporó a ELECTRONICA la capacidad de **delegar subtarefas a subagentes especializados con proveedor distinto**, replicando el seam “*un contrato, muchos proveedores*” del Agent Harness DeepSeek (dsh) pero manteniendo el control determinista del proyecto:

- **Decisión determinista:** `execution/enrutador.py` -delegacion decide el subagente destino por reglas puras (SUBAGENTS: flash→sub-rutina, deepseek→sub-sintesis, opus→sub-critico).
- **Subagentes opencode:** tres agentes en `.opencode/agents/` SIN modelo fijo (Opción A): heredan el motor rotativo del asistente y no consumen créditos OpenRouter.
- **Trazabilidad append-only:** vocabulario de `execution/sesion_log.py` ampliado con `delegacion/decidida` y `delegacion/resultado`.
- **Prueba en vivo:** delegación real de un resumen a sub-rutina vía Task tool, validada y trazada de punta a punta (4 eventos, cadena de hashes íntegra).

Se respetó el guardrail de determinismo: ninguna decisión de tier/subagente se tomó razonando en el chat; solo se extrajo el descriptor y se ejecutó el enrutador.

1 Contexto y motivación

👁 Contexto Al examinar el DeepSeek Harness como parte de la evaluación de sus características (sesiones previas: `Sessions/2026-09-08_exploracion_dsh.md`, `memo docs/AGENTE_IA/dsh_evaluacion_incorporacion.tex`), se detectó que dsh **delega trabajo a subagentes de distintos proveedores** mediante un punto de extensión único. Esa observación motivó esta sesión: entender el mecanismo a fondo, mapearlo a lo que opencode/ELECTRONICA ya ofrece y decidir cómo incorporarlo sin romper la arquitectura de 3 capas.

Por qué es relevante: el proyecto ya incorporó la trazabilidad append-only de dsh (commit 45eaf64), el siguiente candidato natural era la delegación. La diferencia clave es que ELECTRONICA es un espacio determinista: el mismo descriptor debe producir siempre la misma elección de tier y de subagente, mientras que en dsh la elección del proveedor/modelo puede quedar en manos del modelo.

2 Cómo delega dsh: el seam `ctx.subagents`

 Mecanismo en dsh El harness expone un servicio de registro de proveedores bajo el nombre `ctx.subagents`, siguiendo un contrato **Service Definition / Provider / Consumer**. Cualquier plugin puede registrar un proveedor de subagentes; la herramienta model-facing `tool-subagent` queda disponible *una por proveedor*, cada una con su propio `toolName`.

Backends de proveedor en dsh

Backend	Tipo / cómo se ejecuta
<code>subagent-spawn-in-process</code>	in-process; hijo nuevo que hereda el historial del padre
<code>subagent-fork-in-process</code>	in-process; hijo sembrado desde el historial completado
<code>subagent-acp</code>	out-of-process; hijo con su propio runtime vía protocolo ACP
<code>subagent-codex</code>	out-of-process; Codex real (app-server)
<code>subagent-claude-code</code>	out-of-process; Claude Code real (Agent SDK)
<code>subagent-dsh-sdk</code>	out-of-process; harness completo vía SDK

One contract, many providers

El diseño de dsh separa **el contrato de delegación** (cómo se invoca y se devuelve el resultado) de **los proveedores concretos**. Esto permite componer un harness que mezcla subagentes in-process con herramientas externas reales (Codex, Claude Code), sin que el modelo ni la capa de orquestación dependan del backend. Además `tool-subagent-control` ofrece `send_message`, `interrupt_agent` y `list_agents` para hijos *continuales* (durables por id y reanudables).

Traducción a ELECTRONICA: en nuestro caso el rol del `tool-subagent` lo cumple el **Task tool** de opencode (subagentes natives: `general`, `explore`, etc. más los nuestros). La selección del destino, en vez de quedar en manos del modelo, la decide **un script determinista** (el enrutador). Los *providers* de opencode no se registran en un seam: se definen como agentes en `.opencode/agents/`, con permisos y persona por tier. No se incorporó mensajería de hijos continuables (no necesario hoy); sí se trazó cada delegación en el log append-only.

3 Diseño en ELECTRONICA: tres capas

 Arquitectura Como exige el marco del proyecto, la nueva capacidad se distribuyó en las 3 capas: Directiva (SOP), Orquestación (opencode) y Ejecución (scripts deterministas).

3.1 Capa 1 – Directiva: `directives/delegacion_subagentes.yaml`

 SOP Define el *qué*: delegar subtarefas a subagentes especializados según un descriptor estructurado, de forma determinista. Contiene:

- **Goal:** la decisión del subagente destino vive en `enrutador.py`, nunca en el chat.
- **6 pasos:** extraer descriptor → invocar `enrutador.py -delegacion` → registrar `delegacion/decidida` → invocar el Task tool con `subagent_type = <subagent>` → validar salida (retry máx 3, fallback por cadena) → registrar `delegacion/resultado`.
- **Tabla tier→subagente:** flash→sub-rutina (mecánico), deepseek→sub-síntesis (contexto masivo), opus→sub-crítico.
- **Edge cases:** modelo explícito (subagent null), tipo fuera del vocabulario (código 1), subagente que no devuelve resultado válido, subagente que intenta decidir el tier (prohibido), LaTeX con pitfalls documentados.

3.2 Capa 2 – Orquestación (opencode)

⚙️ Orquestador La capa de decisión sigue siendo abierta (no determinista en su naturaleza probabilística), pero su *relación* con la elección de subagente es estricta: el orquestador **extrae el descriptor** de la petición (parsing de intención) y delega la decisión en el enrutador. Luego invoca el subagente con el **Task tool** pasándole la subtarea, contexto y restricciones de salida. No elige modelo ni subagente por criterio subjetivo.

3.3 Capa 3 – Ejecución (scripts deterministas)

🔧 Cambios Tres piezas de código cambian:

1. **execution/enrutador.py:** constante `SUBAGENTS` (`{"flash": "sub-rutina", "deepseek": "sub-síntesis", "opus": "sub-crítico"}`), `DELEGACION_REASON` y el flag `-delegacion`, que añade al JSON de salida `subagent` y `delegacion_reason`. Con `-modelo-explicito` el subagente sale null (elección del orquestador; solo el modelo es override).
2. **execution/sesion_log.py:** vocabulario ampliado con `delegacion/decidida` y `delegacion/resultado`.
3. **.opencode/agents/{sub-rutina, sub-síntesis, sub-crítico}.md:** tres subagentes opencode (mode: subagent) sin modelo fijo.

Una traza mínima de la cadena append-only registrada en la prueba en vivo se muestra a continuación (secuencias 1–4, cada evento con su hash prev encadenado):

```
1 (seq 1) flujo/inicio           {flujo: delegacion_subagentes, subtarea: resumen}
2 (seq 2) delegacion/decidida    {task: resumen, tier: flash, subagent: sub-rutina,
3                                tokens: 1957, critico: false, vision: false}
4 (seq 3) delegacion/resultado    {subagent: sub-rutina, status: ok, exit_code: 0}
```

```
5 (seq 4) flujo/fin {status: ok, exit_code: 0}
```

4 Los logs append-only en profundidad

≡ Trazabilidad La trazabilidad que acompaña a la delegación reposa en `execution/sesion_log.py`, la pieza de trazabilidad append-only incorporada a ELECTRONICA (ideal adaptado de dsh: *append-only event log como fuente de verdad*). Esta sección explica el mecanismo, porque es la base sobre la que se apoyan los eventos `delegacion/*`.

4.1 Principio: el log nunca se modifica ni se borra

La idea central es que cada evento se **ACUMULA** en un archivo JSONL (`.tmp/session_log-<run>.jsonl`) y **jamás se altera ni elimina**. El log es la *fuente de verdad*; cualquier estado que necesites (checkpoints, reanudación, métricas) es una **vista derivada** que se calcula leyéndolo. Esto hace que la historia completa sea auditable y recuperable aunque un paso intermedio falle.

4.2 Estructura de cada evento

Cada línea del log es un evento JSON con esta forma:

```
1 {
2   "seq": 3,                # numero de secuencia (crece con cada
                             # evento)
3   "ts": "2026-09-09T13:46:24.473272+00:00", # marca de tiempo UTC
4   "run": "test_vivo",      # identificador de sesion/ejecucion
5   "tipo": "delegacion/resultado", # vocabulario controlado
6   "datos": {"subagent": "sub-rutina", "status": "ok", "exit_code": 0},
7   "prev": "8c3ee112...62ba", # hash SHA-256 de la línea anterior (
                             # cadena)
8   "append_only": true      # sello de inmutabilidad
9 }
```

4.3 Cadena de hashes: la garantía de integridad

El campo `prev` contiene el **SHA-256 de la línea anterior completa**. Al leer el log, cada evento debe cumplir: su `seq` es el consecutivo correcto y su `prev` coincide con el hash de la línea previa real. Cualquier edición, borrado o reordenamiento (por ejemplo, cambiar `"seq": 1` a `"seq": 99`) rompe la cadena en el punto alterado. El subcomando:

```
1 python3 execution/sesion_log.py integrity --run test_vivo
```

recorre toda la cadena y devuelve ok: true si está íntegra, o **código de salida 2** con el evento conflictivo si detecta manipulación. Este fue el mecanismo verificado en la prueba en vivo (4 eventos, cadena íntegra) y en el banco de pruebas (manipulación simulada detectada con exit 2).

4.4 Vocabulario controlado de tipos de evento

Los tipos no son libres: se validan contra TIPOS_EVENTO en `sesion_log.py`. Invocar add con un tipo fuera del vocabulario falla con código 1. El vocabulario actual es:

TIPOS_EVENTO

Tipo	Qué registra
flujo/inicio	Arranque de un flujo multi-paso
flujo/paso	Paso completado ({paso, script, status})
flujo/fin	Flujo terminado correctamente
flujo/error	Fallo ({paso, script, mensaje})
tool/call	Invocación de herramienta/script ({script, args})
tool/result	Resultado de herramienta ({status, exit_code})
delegacion/decidida	Decisión del enrutador ({task, tier, subagent, tokens, critico, vision})
delegacion/resultado	Resultado del subagente ({subagent, status, exit_code})

4.5 Checkpoints y reanudación como vistas derivadas

`sesion_log.py state` deriva el checkpoint `current_step`, `pasos_completados` y errores a partir del log (nunca al revés); `resume` informa si un run es reanudable y desde qué paso según su último evento (`fin` → no reanudable; `error` → reanudable desde el paso fallido; `en_curso` → siguiente paso pendiente).

4.6 Defensas adicionales

- **run_id sanitizado:** solo [A-Za-z0-9._-], máx 120 caracteres; un `run_id` malicioso (p.ej. `../etc/evil`) es rechazado.
- **Cierre de línea:** si el archivo no termina en salto de línea, se sospecha una escritura interrumpida y add falla avisando de ejecutar `integrity`.
- **Serialización canónica:** cada evento se escribe con claves ordenadas (`sort_keys`), lo que hace la cadena de hashes reproducible.

En resumen: *append-only* + *hash chain* + *vocabulario controlado* + *run_id* sanitizado. Los eventos delegacion/decidida y delegacion/resultado quedan así protegidos por el mismo mecanismo que el resto de la trazabilidad: una vez escritos, son inmutables y cualquier manipulación posterior es detectable.

5 Decisiones de diseño (aprobadas por el usuario)

✔ Decisiones Cuatro acuerdos tomados antes de implementar:

✔ Decisiones de la sesión

Tema	Decisión
Modelo de los subagentes	Opción A: sin modelo fijo; heredan el motor rotativo del asistente (0 créditos OpenRouter). La especialización es por persona y permisos.
-modelo-explicito	El subagente sale <code>null</code> ; la elección del destino queda a criterio del orquestador (solo el modelo es <code>override</code>).
Trazabilidad	Ampliar el vocabulario de <code>sesion_log.py</code> con tipos <code>delegacion/*</code> .
Artefactos <code>.tmp/</code>	No tocar los pendientes de sesiones anteriores (<code>run_state.json</code> , clones, etc.).

El motivo de la **Opción A** es mantener el espíritu económico del proyecto: el motor del asistente rota entre modelos Free (Zen/OpenRouter) y **nunca consume** `OPENROUTER_API_KEY`. Si en el futuro se quisiera fijar modelo por tier (Opción B), cada invocación del subagente consumiría créditos (especialmente opus, \$5/\$25 por millón de tokens), por lo que se evaluaría el saldo antes de activarlo.

6 Verificación (0 créditos)

🔧 Pruebas La verificación se hizo sin coste alguno de API:

>_ Salidas reales del enrutador

Al ejecutar `execution/enrutador.py` con distintos descriptores y `-delegacion -no-log`:

- `-task formateo -tokens 1000` → tier flash, subagente: sub-rutina
- `-task contexto_masivo -tokens 60000` → tier deepseek, subagente: sub-sintesis
- `-task examen_complejo -critico -tokens 9000` → tier opus, subagente: sub-critico
- `-task debug -modelo-explicito moonshotai/kimi-k3` → tier explicito, subagente: null

- `-task nodo_inventado -tokens 5` → error código 1 (vocabulario)

Determinismo: se ejecutó el mismo descriptor dos veces y ambas salidas fueron idénticas (compare `[A = B]`).
sesion_log: cadena completa (inicio → decidida → paso → resultado → fin) con integrity OK; la manipulación de la secuencia (seq 1→22) fue detectada con código de salida 2.

7 Prueba en vivo (tras reinicio de opencode)

⚡ End-to-end El usuario reinició opencode para cargar los 3 subagentes y se ejecutó la prueba completa siguiendo la directiva:

1. Descriptor: `-task resumen -archivos directives/delegacion_subagentes.yaml` (1957 tokens estimados).
2. `enrutador.py -delegacion` → tier flash, subagent: sub-rutina.
3. `sesion_log.py add` → flujo/inicio (seq 1) y delegacion/decidida (seq 2).
4. **Task tool** con subagent_type: sub-rutina: el subagente leyó el archivo y devolvió un **resumen JSON válido** (título, propósito, 6 pasos, 3 subagentes, 5 edge cases) sin editar archivos (su permiso de edición es deny).
5. Validación del JSON contra lo solicitado: OK.
6. `sesion_log.py add` → delegacion/resultado (seq 3, exit 0) y flujo/fin (seq 4).
7. integrity → cadena de hashes íntegra en los 4 eventos.

Resultado: la delegación determinista funciona de punta a punta y queda trazada en el log append-only. Se emitió la alerta audible de completado (`execution/alert_user.py success`) y el log de prueba fue eliminado (`.tmp/session_log_test_vivo.jsonl`).

8 Comandos de referencia

> CLI Los comandos clave utilizados:

```
1 # 1. Decidir el subagente (determinista, 0 credits, sin log para pruebas)
2 python3 execution/enrutador.py --task resumen \
```

```

3  --archivos directives/delegacion_subagentes.yaml --delegacion --no-
   log
4
5  # 2. Trazar inicio y decision de delegacion
6  python3 execution/sesion_log.py add --run test_vivo --tipo flujo/inicio
   \
7  --datos '{"flujo":"delegacion_subagentes","subtarea":"resumen"}'
8  python3 execution/sesion_log.py add --run test_vivo --tipo delegacion/
   decidida \
9  --datos '{"task":"resumen","tier":"flash","subagent":"sub-rutina","
   tokens":1957}'
10
11 # 3. (Orquestador) Task tool -> subagent_type: sub-rutina (delegacion)
12
13 # 4. Trazar resultado y cierre
14 python3 execution/sesion_log.py add --run test_vivo --tipo delegacion/
   resultado \
15 --datos '{"subagent":"sub-rutina","status":"ok","exit_code":0}'
16 python3 execution/sesion_log.py add --run test_vivo --tipo flujo/fin \
17 --datos '{"status":"ok","exit_code":0}'
18
19 # 5. Verificar integridad de la cadena
20 python3 execution/sesion_log.py integrity --run test_vivo

```

9 Pendientes y líneas siguientes

 Siguiendo Quedan anotados como pendientes:

- Ampliar SUBAGENTS con más subagentes si aparece un nuevo rol (editar solo el mapa de execution/enrutador.py + re-test).
- Opcional (futuro): fijar modelo por tier (Opción B) previa evaluación de saldo, dado el consumo de créditos que implicaría.
- Instalar en la práctica: probar una delegación de tarea crítica (opus) a sub-critico para cubrir ese camino de la matriz.

Guardrail respetado: en ninguna decisión de esta sesión se eligió tier o subagente razonando en el chat. La política vive en execution/enrutador.py (matriz SUBAGENTS) y en la directiva delegacion_subagentes.yaml; afinar umbrales se hace con telemetría ('tmp/routing_log.jsonl'), no a ojo.

Referencias

- `directives/delegacion_subagentes.yaml` (SOP de la capacidad)
- `execution/enrutador.py` (decisión determinista de tier/subagente)
- `execution/sesion_log.py` (trazabilidad append-only)
- `.opencode/agents/sub-rutina.md`, `sub-sintesis.md`, `sub-critico.md`
- `Sessions/2026-09-09_delegacion_subagentes.md` (bitácora de la sesión)
- `.agent/enrutamiento.md` (sección “Delegación a subagentes”)
- Repo clonado de dsh: `.tmp/repo_clone_deepseek-ai-deepseek-harness/packages/subagent/`